

WEEK 8 | SCALABLE TRAINING + DEPLOYMENT

Deployment Engineering on Kubernetes

Satyadhyan Chickerur, Ph.D., MBA(Edu.Mgmt), SMIEEE
Director, Centre for Artificial Intelligence Research,
Prof., School of Computer Science & Engineering,
KLE Technological University,
BVB Campus, Vidyanagar,
Hubballi - 580031, Karnataka, India

The Big Picture — Two Different Questions

You've packaged a great model . Now: how do you actually run it in production, and keep it running when things go wrong?

Acceleration

Does it run fast?

Kernel fusion, quantization, serving engines like vLLM/Triton.

Deployment

Does it operate reliably?

Rolling updates, probes, autoscaling, canary rollout, SLOs, CI/CD.

THINK OF IT LIKE THIS: The fastest race car in the world is useless without brakes, mirrors, and a pit crew. Acceleration makes the model fast; deployment engineering makes it SAFE to run — reversible, observable, and recoverable when it breaks.

Key Terms Before We Start

A quick shared vocabulary — skip ahead if these are all second nature to you.

Rollout — The process of replacing an old running version of a service with a new one.

Probe — A Kubernetes health check that decides if a container is ready or alive.

Canary deployment — Sending a small slice of traffic to a new version before fully switching over.

SLO (Service Level Objective) — An explicit promise about how the service should perform, e.g. '99.9% under 500ms.'

Rollback — Reverting to the previous working version after a bad deploy.

CI/CD — Automated pipelines that test and ship changes without manual, error-prone steps.

The Inference Pod — Annotated Manifest

```
spec:
  replicas: 3
  strategy: { type: RollingUpdate, rollingUpdate: { maxSurge: 1, maxUnavailable: 0 } }
  containers:
    - image: ghcr.io/team/llm-inference:2.3.1
      env: [{ name: MODEL_URI, value: 'my-llm@sha256:abc123...' }]
      startupProbe: { httpGet: { path: /healthz }, failureThreshold: 60, periodSeconds:
5 }
      readinessProbe: { httpGet: { path: /readyz }, periodSeconds: 5 }
      resources:
        requests: { cpu: '2', memory: '16Gi', nvidia.com/gpu: 1 }
        limits: { cpu: '4', memory: '24Gi', nvidia.com/gpu: 1 }
```

Strategy

maxUnavailable=0: never drop a working pod.

Digest + startup

Model pinned by SHA. 5-min grace before liveness.

Resources

requests == limits on GPU — required for Guaranteed QoS.

Four Rollout Strategies — When Each Fits

Rolling update

Replace pods N at a time, K8s-native. Default for most stateless services.

Blue/green

A full new stack, flip a switch. Instant rollback, but 2x resources briefly.

Canary

Route 1-5% of traffic to the new version, watch metrics, ramp up.

Shadow

Duplicate requests to the new version, compare outputs, don't serve them.

RECOMMENDATION: rolling for minor bumps; canary + shadow for model changes; blue/green when you need instant rollback.

SLOs and the Four Golden Signals

Latency

p50/p95/p99 request time. For LLMs: time-to-first-token + inter-token.

Traffic

QPS, tokens/s. Model services often scale on tokens, not requests.

Errors

5xx rate + silent failures. Add synthetic checks for quality drops.

Saturation

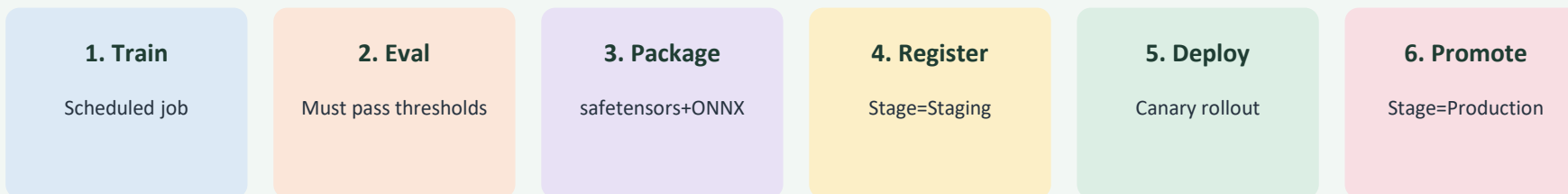
GPU util, queue depth, KV cache usage.

SLO = EXPLICIT PROMISE + ERROR BUDGET

Example: 99.9% of requests return within 500ms over 28 days. Error budget = 0.1% = ~43 min of 'bad' per month.

If you blow the budget: freeze feature work, fix reliability.

CI/CD for Models — the Training->Serve Pipeline



```
# GitHub Actions — eval FAILS the job if score drops; nothing broken reaches Staging
on: { schedule: [{ cron: '0 6 * * 1' }] }
jobs: { eval: { steps: [{ run: 'python evaluate.py --min-score 0.82' },
  { run: 'mlflow models register ... --stage Staging' }] } }
```

The eval gate is the safety valve: a pipeline that retrains but promotes blindly will eventually ship a regression. Never skip it, even under deadline pressure.

Rollback Playbook — When a Deploy Goes Wrong

Every rollout strategy needs a matching rollback plan decided BEFORE the incident, not during it.

1 Detect

SLO alert fires: error spike or latency breach.

2 Decide

Is it the new model or infra? Compare canary vs baseline.

3 Rollback

Change the image digest back — should take seconds.

4 Review

Was the eval gate or canary window insufficient? Fix the gate.

Digest-pinned deploys are what make step 3 fast — rollback is changing one string, not rebuilding anything.

Common Mistakes & How to Spot Them

Mistake	How it shows up	Fix
No startupProbe on slow-loading models	Pods crashloop forever right after deploy	Add startupProbe with a generous failureThreshold
Skipping the eval gate under pressure	A regression reaches production silently	Never let CI promote without a passing eval
Using human tags instead of digests	Can't say exactly what's running, rollback is slow	Pin images and models by SHA digest
No SLOs defined at all	Nobody can say whether the service is 'healthy'	Define latency/error/saturation SLOs up front

Quick Recap — Terms and Concepts

Rolling / canary / blue-green / shadow — Four rollout strategies, each with a different risk/cost trade-off

Probes — startupProbe for slow model load; readinessProbe to gate traffic

SLO + error budget — An explicit promise, plus a quota of allowed 'bad' before you must slow down

CI/CD eval gate — The safety valve that blocks a regression from ever reaching Staging

Rollback — Fast and safe only if images/models are digest-pinned, not tag-pinned

Week 8 Recap

1

Scale LR as global batch grows

2

Checkpoint correctly and recover from failures

3

Package a model as a trustworthy shipping artifact

4

Deploy and operate it safely on Kubernetes